

Target RLDS Dataset Schema

Source data: LeRobot dataset `test_name_20260910_v05`, collected via keyboard teleoperation of a simulated UR5e (URSim), 8 episodes, 1708 total frames, 15 fps.

This document describes how the recorded LeRobot data will be mapped into RLDS’s episode/step structure, per Section 3.5 of the task description.

1. Episode Structure

Each **episode** = one continuous teleoperated demonstration, bounded by the recorder’s own episode start/reset markers.

RLDS field	Source	Notes
Episode boundary	LeRobot <code>episode_index</code>	8 episodes (0-7) in the current dataset
<code>is_first</code>	Derived: <code>frame_index == 0</code> within an episode	First frame of each episode
<code>is_last</code>	Derived: last <code>frame_index</code> within an episode	Last frame before reset
<code>is_terminal</code>	Same as <code>is_last</code> for this dataset	No distinct “failure” vs. “success” termination is tracked — teleop episodes end on user reset, not task completion
Episode metadata	<code>task_index</code> → task description string	Currently a single task (“test_name”); real task descriptions will be added when meaningful demonstrations are collected

2. Step Structure

Each **step** = one recorded frame (15 fps), containing a synchronized observation, action, and timestamp.

2.1 observation

Field	Source (LeRobot)	Shape / dtype	Notes
<code>robot_state</code>	<code>observation.state</code>	<code>float32[48]</code>	See breakdown below
<code>image_wrist</code>	<code>observation.images.wrist_image</code>	<code>uint8[480, 640, 3]</code>	Currently a dummy placeholder frame (no camera attached yet); decoded from stored AV1 video at load time
<code>image_exterior</code>	<code>observation.images.exterior_image</code>	<code>uint8[480, 640, 3]</code>	Same as above — dummy until a real/rendered camera is added (Section 3.3 of the task)

observation.state (48-dim) breakdown:

Sub-field	Dims	Description
<code>tcp_pose</code>	6	End-effector pose: x, y, z, rx, ry, rz
<code>tcp_speed</code>	6	End-effector velocity
<code>tcp_force</code>	6	Force/torque at end-effector
<code>tcp_acc</code>	3	End-effector linear acceleration (x, y, z)
<code>gripper</code>	3	<code>raw_position</code> , <code>raw_bin</code> , <code>action_bin</code>
<code>joint.pos</code>	6	Joint positions (rad), one per UR5e joint
<code>joint.vel</code>	6	Joint velocities
<code>joint.acc</code>	6	Joint accelerations
<code>joint.force</code>	6	Joint torques

2.2 action

Field	Source	Shape / dtype	Notes
<code>action</code>	<code>action</code>	<code>float32[6]</code>	Cartesian delta command: <code>delta_x</code> , <code>delta_y</code> , <code>delta_z</code> , <code>delta_rx</code> , <code>delta_ry</code> , <code>delta_rz</code>

		<code>delta_rx</code> , <code>delta_ry</code> , <code>delta_rz</code> — the keyboard-teleop step deltas sent to the robot each control cycle
--	--	--

2.3 reward

Not used. This is a pure imitation-learning data collection task; no reward signal is computed or stored. RLDS's `reward` field will be omitted or set to `0.0` as a placeholder if the schema requires it structurally.

2.4 Per-step metadata

Field	Source	Notes
<code>timestamp</code>	<code>timestamp</code>	Float seconds, used for synchronization verification
<code>frame_index</code>	<code>frame_index</code>	Frame position within its episode
<code>episode_index</code>	<code>episode_index</code>	Which episode this step belongs to
<code>global_index</code>	<code>index</code>	Global frame index across the whole dataset
<code>task_index</code>	<code>task_index</code>	Maps to the task description string (language instruction, if/when used)

3. Synchronization Notes (Section 3.9)

- **Robot control rate:** RTDE state reads occur at the robot’s native control frequency; teleop steps are logged at the dataset’s fixed **15 fps**.
- **Camera frame rate:** Currently 15 fps (matched to robot logging rate) using dummy frames; a real/rendered camera will need to be resampled or matched to this same rate.
- **Timestamp generation:** Each step’s `timestamp` is recorded at the moment the frame is logged by the LeRobot recording loop, alongside the corresponding robot state read and action command — all three are captured within the same loop iteration, ensuring per-step alignment by construction rather than post-hoc matching.
- **Missing/delayed frames:** Not yet encountered with dummy cameras (which never fail to produce a frame). This will need explicit handling once a real webcam is introduced — planned approach: on a missing frame, repeat the last successfully captured frame and flag it, rather than dropping the step (to preserve fixed-rate alignment with robot state).

4. Open Questions for Bahaeddine

- Should camera images be included in the RLDS conversion now (as dummy frames), or should the schema/conversion script be written to wait until a real camera is integrated?
- Is a placeholder `reward = 0.0` needed for RLDS/TFDS tooling compatibility, or can the field be omitted entirely?
- Should `is_terminal` distinguish “task completed” from “manually reset,” or is treating every episode end as terminal acceptable for this dataset’s purpose?